

Тема 1

- Алгоритъм (интуитивно понятие) – Последователност от инструкции (стъпки), които се изпълняват, за да се реализира конкретна цел/задача, той описва логиката, по която се реализира конкретната функционалност или се решава дадената задача. Тази логика може да бъде представена по различни начини – словесно (с думи), графично (блок-схема) и чрез език за програмиране (код).
- Свойства:
 - Определеност – При едни и същи данни резултата е строго определен;
 - Крайност – Изпълнява се от определен брой стъпки;
 - Дискретност – Процесът на работа е разделен на отделни, последователни стъпки във времето;
 - Множество от входни данни (деф. област) – данни за които алгоритъма работи коректно;
- Характеристики
 - Резултатност
 - Масовост
 - Цикличност
- Видове алгоритми според:
 - Структурата:
 - Линейни – стъпките се изпълняват в строга последователност
 - Разклонени – Условни, делят се на циклични и нециклични
- Алгоритъм на Евклид (НОД) – използва цикличен подход:
 1. Делим първото число M на второто N и взимаме остатъка r
 2. Изместваме числата – делителя става делимо, а остатък – делител
 3. Това се повтаря, докато остатък стане 0
 4. Последното число, на което сме делили преди r да стане 0 е НОД
- Понятие за програма и езици за програмиране
 - Програми = Алгоритми + Структури от данни
 - Докато алгоритъмът е логиката за решаване на задачата, програмата е нейната конкретна реализация, записана чрез език за програмиране.
 - Езиците за програмиране биват от ниско ниво (машинен или асемблерен) и от високо ниво (императивни и приложни)

Тема 2

- Сложност на алгоритъм $O(f)$ – Оценка, показваща как нараства броят на стъпките за изпълнение на програмата, при увеличаване на количеството входни данни. Тя бива:
 - Константна $O(1)$ – За изпълнението са необходими константен брой стъпки, независимо от обема на входните данни
 - Линейна $O(N)$ – Броят на необходимите стъпки е приблизително равен на броя елементи. Времето расте право пропорционално с данните
 - Квадратична $O(n^2)$ – Броят на необходимите стъпки е равен на (броя на входните данни)²
 - Кубична $O(n^3)$ – Броят на необходимите стъпки е равен на (броя на входните данни)³
 - Логаритмична $O(\log N)$
 - Експоненциална $O(2^n)$
- Времева сложност $T(n)$ - Функцията, показваща колко време (операции) са нужни за изпълнение на алгоритъма спрямо размера на задачата. Тя бива:
 - Минимална T_{min} – Времето, за което алгоритъмът приключва работа в най-добрия възможен случай при един и същ размер на задачана.
 - Средна T_{cp} – Средното време за изпълнение, получено, като се вземат предвид всички възможни случаи.
 - Максимална T_{max} – Времето за изпълнение на алгоритъма при най-неблагоприятните входни данни. (най-лошият възможен случай)
 - Асимптотична – Функция, показваща скоростта на нарастване на времето за изпълнение при много големи стойности на входните данни, като пренебрегва константите и по-малките членове.
- Производителност – Скоростта, с която компютърната система обработва данните. Зависи от технологията и компютърната архитектура и се оценява в милиони инструкции за секунда (MIPS), милиони операции за секунда (MOPS) и милиони операции за числа с плаваща запетая в секунда (MFLOPS).
- Влияние върху производителността – Скоростта на изпълнение зависи от сложността на алгоритъма. Колкото е по-малка, толкова по-бързо работи.
- Розмер на задачата (n) – Количеството входни данни на даден алгоритъм.
- Пространствена сложност – Функцията, показваща количеството памет (RAM) нужна за изпълнението на алгоритъма.
 - P-спасе – Задачи, които изискват полиномиална памет (нормално количество)
 - Exp-спасе – Задачи, които изискват експоненциална памет (огромно количество)
- Класификация на задачите
 - P-задачи – Задачи, които могат да бъдат решени бързо.
 - NP-задачи – Задачи чието решение може да бъде проверено бързо (полиномиално време), но намирането му да е бавно
 - Експоненциални задачи – изискват много време, а често и памет

Тема 3

- Структура от данни – Начин на организация, определящ типа на данните, взаимното им разположение и начина на достъп до тях. Делят се на два основни типа:
 - Атомарни (прости) – Реални, целочислени, символни и логически
 - Съставни (структурирани) – Според това дали могат да променят размера си се делят на статични (масив, множество, запис) и динамични (линейни и нелинейни). Линейните биват стек, опашка, дек, списък, а нелинейните – дърво или граф.
- Масив – Статичен тип данни представляващ подредена структура от краен брой еднотипни данни (елементи). Размерът му е фиксиран, а достъпът до елементите му се осъществява чрез индекс. Бива едномерен и двумерен.
- Статични структури от данни – Структури с предварително определен и фиксиран брой елементи, който не се променя по време на работа. Използваната оперативна памет е също с фиксиран обем, което не винаги се използва ефективно. Тя се заделя още при компилирането на програмата. Този тип структури са трудни за обновяване (добавяне или премахване на елементи) и имат ограничение на капацитета, което не може да се надвишава.
- Динамични структури – Създават се и унищожават по време на изпълнение на програмата, а използваната памет се променя динамично в зависимост от големината на структурата (нараства при добавяне и намалява при изтриване на елемент). При тях имаме три основни действия (запис, четене и търсене)
- Линейни структури – Всеки елемент има най-много два съседни елемента (предишен и следващ). Само първият и последният елемент имат по един съседен. Към тях спадат стек, опашка, дек и списък.
- Стек LIFO – Линейна структура, представляваща подредена последователност от еднотипни елементи, записът и четенето, на които се извършва от върха. Работи на принципа, че последният влязъл е първият излязъл.
- Опашка FIFO – Линейна структура, при която операциите се извършват в двата края (начало и край). Принципът на работа, е че първият влязъл е първият излязъл. Съществува и така наречената опашка с приоритети. При нея изваждането на елементи става в някакъв ред. Това може да се реализира като поддържахме опашката сортирана.
- Дек (Deque - Double-Ended Queue) – Опашка, в която могат да се добавят и премахват елементи и от двата края. Може да има ограничен вход (елементи се премахват от двата края, но вмъкването е само от единия) или ограничен изход (може да се вмъква от двата края, но да се премахва от единия)
- Лист / списък – Записът и четенето могат да се правят на произволно място. Биват едноточни (всеки елемент има един указател към следващия) или двуточни (всеки елемент има два указателя – към следващия и предишния)

Тема 4

- Граф – Нелинейна динамична структура от данни изградена от възли (върхове) и връзки между тях наречени ребра (клони). Нагледно се изобразява като множество от точки (върхове / възли), свързани помежду си с линии (ребра / клони). Всеки връх може да бъде начален в зависимост от задачата. Ребрата могат да бъдат паралелни (две или повече ребра свързват директно два върха) или бримка (ребро, за което началния и крайния възел съвпадат) Има два типа граф:
 - Ориентиран – Дъгите (ребрата), свързващи възлите са едноразсочни
 - Неориентиран – Ребрата /клоните, свързващи възлите са двуразсочни
- Степен на възел – Броят клони, свързани с възела
 - Изолиран възел – степента му е 0
 - Висящ възел – степента му е 1
- Мултиграф – Граф, в който се допускат паралелни ребра
- Път – Последователност от върхове, в която всеки два последователни върха са свързани с ребро от графа
- Цикъл (контур) – Път, при който началния и крайния възел на последователността съвпадат
- Прост цикъл – Няма повтарящи се върхове освен началния и крайния
- Хамилтонов цикъл – Включва всички върхове на графа точно по веднъж
- Ойлеров цикъл – Включва всички ребра на графа точно по веднъж
- Видове графи:
 - Пълен – Всеки връх е свързан със всеки
 - Празен граф – Върховете не са свързани
 - Допълнителен граф – Съдържа липсващите ребра от оригиналния (допълва го)
 - Регулярен (еднороден) граф – Всички върхове имат еднаква степен
 - Тегловен граф - граф, на чиито възли или/и клони има присвоени тегла
 - Равнинен граф – Може да се начертае в равнината без ребрата му да се пресичат, освен във върховете
 - Свързан граф – Между всеки два върха съществува път
 - Несвързан граф – Състои се от няколко отделни части, между които няма път
 - Подграф – Част от граф, съставена от подмножество от неговите върхове и ребра
- Представяне на граф
 - Външно – В текстов файл описваме структурата на графа. Има два начина:
 - Списък със сепаратор – Записва се номера на възела и след тирето се изреждат неговите съседи, а с # обозначаваме края на задаването: 1-2 3 2-3 4 5 3-1 2 4 5 #
 - Позиционно записване по редове – На всеки ред се записват само съседните възли, чийто номер отговаря на този на реда. Ако възела няма съседи (изолиран) на прилежащия му ред се записва съответно 0 #
 - Тегловен граф – След номера на съседния възел се записва теглото на клона: 1-2 (50) 3 (75) 3-4(21) 5(34)#

- Насочен (ориентиран) граф – За всеки възел се записват само или входящите, или изходящите клони
- Вътрешно – Структурата от данни, която програмата създава в оперативната памет, за да може ефективно да обработва връзките между обектите. Извършва се по два начина:
 - Матрица на съседство (двумерен масив) – Табличен модел, при който бързо се проверява дали два възела са свързани. Редовете и колоните съответстват на номерата на възлите в графа и записваме 1 в съответната клетка, ако между двата възела има връзка и 0, ако няма, като ако няма бримки в клетките на главния диагонал трябва да стоят изцяло нули.
 - Списъци на съседство – Списъчен модел, при който се съхраняват само съществуващите връзки с цел икономия на памет
- Построяване на граф – Динамичен процес за създаване и конфигуриране на граф в оперативната памет. Представлява поредица от стъпки, чрез които се дефинира броя на върховете и се установяват връзките (ребрата) между тях. Основните операции за построяване на граф са:
 - Създаване на празен граф – Инициализира се празна динамична структура от данни, която предстои да бъде попълвана с върхове и връзките между тях
 - Добавяне / премахване на връх – Процес, при който се създава или заличава обект в графа чрез разширяване или намаляване на използваната структура от данни. При премахване на връх системата трябва автоматично да изтрие и всички свързани с него ребра.
 - Добавяне / премахване на ребро – Операция за установяване или прекъсване на директна връзка между два съществуващи върха чрез промяна на съответната стойност в структурата
 - Проверка за съществуване на връх – Операция, проверяваща дали даден номер на връх е валиден и включен в графа. Става чрез проверка дали индекса на възела е в границата на структурата.
 - Проверка за съществуване на ребро – Операция, установяваща дали има връзка между два конкретни върха.
 - Намиране на наследници - Процес на извличане на всички върхове, до които има директен път от текущия възел. Това става чрез преглед на неговия ред в матрицата или на съответния му списък на съседство.
- Обхождане в ширина – Обхождане на граф, при което се посещават всички преки съседи на началния възел, после техните съседи и т.н. Алгоритъмът изследва графа ниво по ниво, отдалечавайки се равномерно от стартовата точка. За изпълнението му се използва опашка, като за да се избегнат безкрайни цикли всеки посетен възел се маркира в отделен масив. Логиката на алгоритъма е:
 - Избираме начален връх, който маркираме като посетен на съответния му индекс (0 -> 1) в масива, който ползваме за проверка. От опашката изваждаме първия елемент и преди да добавим неговите съседи проверяваме дали са записани със стойност различна от 0 в масива за проверка, като ако са не ги добавяме в опашката. Процеса се повтаря до пълното изпразване на опашката

- Обхождане в дълбочина – Алгоритъм за обхождане, който се стреми да се отдалечи максимално бързо от началния връх, следвайки един път докато не достигне връх без непосетени съседи (край на пътя). Когато пътят се изчерпи, алгоритъмът се връща назад (backtracking) към предходния връх, за да търси нови разклонения. Реализира се чрез Стек (Stack) или Рекурсия.
 - Чрез рекурсия – създава се рекурсивна функция, която извиква себе си за всеки нов непосетен съсед. Когато стигне до задънена улица извиканата функция приключва и се връща един връх назад като извиква сама себе си
 - Чрез стек – Ръчно добавяме всеки новооткрит връх за да навлезем веднага в дълбочина, а връщането назад се реализира чрез премахване (изваждане) на най-горния елемент след установяване, че няма останали необходими съседи.
- Алгоритъм за намиране на минимално скелетно дърво – Има за цел да избере точно определени връзки в графа така че да свърже всички върхове помежду им, като сумата от теглата на връзките да бъде възможно най-малка и да не се образуват затворени кръгове.
 - Алгоритъм на Крускал – Целта му е да избере набор от ребра с минимална сумарна тежест, които свързват всички върхове в графа, без да образуват затворени кръгове (цикли). Алгоритъмът постепенно добавя най-евтините възможни връзки, докато всички върхове не станат част от една структура:
 - Ребрата в графа се сортират във възходящ ред според тяхната тежест
 - Взимаме едно по едно ребрата като за всяко избрано проверяваме дали свързва два върха, които вече са част от едно и също дърво. Ако върховете са в различни компоненти реброто е безопасно, но ако са в една ще получим затворен цикъл
 - Алгоритъмът спира когато успешно сме добавили N-1 ребра
 - Алгоритъм на Дейкстра – Алгоритъм за намиране на най-къс път от един фиксиран начален връх до всички останали върхове в графа. Работи само за графи с положителни тегла на ребрата. Алгоритъмът поддържа списък с "най-краткото известно разстояние" до всеки връх. В началото всички са безкрайност, освен стартовия (0). На всяка стъпка алгоритъмът стъпва в най-близкия непосетен връх и проверява дали през него не може да се стигне по-бързо до съседите му.

Тема 5

- Дърво – Свързан граф без цикли (затворени кръгове), който притежава някои допълнителни свойства и има рекурсивна структура (съставено от по-малки дървета). Дърветата биват неориентирани (ненасочен, свързан граф без цикли и движение в двете посоки) и ориентирани (насочен граф със строга йерархия, при който от корена до всеки друг възел има само един път). Състоят се от следните структури:
 - Върх / възел – Елемент от дървото, който съдържа стойност и може да бъде свързан с други възли
 - Клон / ребро – Връзката между два възела. Показва посоката от един възел към друг
 - Корен – Най-горният възел в дървото, началната точка, от която произлизат всички останали възли
 - Листо – Възел, който няма деца, това са крайните елементи в дървото
 - Родител – Възел, който сочи към един или два възела под себе си
 - Дете / наследник – Възелът, който стои директно под родителя и е свързан към него с клон
 - Поддърво – Структура, която започва от произволен възел и включва всички възли под него
 - Дължина – Броят ребра от един връх до друг
 - Дълбочина – Дължината на дървото от корена до съответния връх или листото
 - Височина – Дължината на дървото от листото до корена или съответния връх
 - Ниво – Поколението, на което се намира възела
 - Степен на възел – Броя на преките наследниците, които има този възел
- Двоично дърво – Подредено ориентирано дърво, при което всеки възел може да има до две деца (ляв и десен наследник). Има рекурсивна структура (състои се от ляво и дясно двоично поддърво)
- Балансирано дърво – Височината на лявото и дясното поддърво се различават най-много с единица. Съществува и идеално балансирано дърво, при него височината на двете поддървета е еднакво. Когато дървото се разбалансира се прилага една от 4-те ротации, за да се балансира отново:
 - Дясна ротация – Когато дървото е натежало отляво (в лявото поддърво на левия наследник) коренът пада едно ниво надолу и заема мястото на десния си наследник измествайки конструкцията на поддървото в дясно, а левия наследник на корена заема неговото място.
 - Лява ротация – Когато дървото е натежало в дясно коренът пада едно ниво надолу в ляво, превръщайки се в ляв наследник, а десният му наследник става новия корен

- Ляво-Дясна ротация – Когато в левия наследник има тежко дясно поддърво първо се извършва лява ротация върху левия наследник и после дясна ротация върху основния корен
- Дясно-Лява ротация – Когато десния наследник има тежко ляво поддърво първо се извършва дясна ротация върху десния наследник и след това лява ротация върху корена
- Двоично дърво за търсене – Структура от данни, организирана по такъв начин, че позволява бързо търсене, добавяне и изтриване на елементи. За всеки възел важи правилото, че левия наследник трябва да е с по-малка стойност от родителя, а десния по-голяма. Това правило позволява търсенето да става с елементарно сравнение. При обхождане винаги се започва от корена през родителите, наследниците и се стига до листата. Когато премахваме елемент трябва да запазим структурата на дървото (ляво дете < по малко от дясно). При изтриване на листо просто премахваме връзката между него и родителя, но ако искаме да изтрием възел с едно дете трябва да свържем родителя му с детето.
- 2-3 дърво – Идеално балансирано дърво за търсене, в което всеки възел (освен листата) може да има два или три наследника, вместо само два. Представява частен случай на B-дървото. В това дърво съществуват два вида възли:
 - Двоун възел – съдържа една стойност и има два наследника. Работи точно като възел в двоично дърво (по-малките са вляво, по големите вдясно)
 - Троен възел – Съдържа две стойности и има три наследника (ляв, среден и десен). Левия наследник е със стойност по малка от тази на родителя, десния с по-голяма, а средния със стойност между тези на батията си.
- 2-3-4 дърво – Идеално балансирано дърво за търсене, в което всеки вътрешен възел може да има два, три или четири наследника. Възлите могат да бъдат три вида – двоен, троен или четворен (има 3 стойности и 4 деца)
- B-дърво – Балансирано дърво за търсене, предназначено за съхранение на големи обеми от данни, където всеки възел може да съдържа множество ключове и да има множество деца. Ако възел съдържа n полета с данни, той има $i=n+1$ наследника.
- Червено – черно дърво – Вид балансирано дърво, но не идеално, при което всеки възел е означен с цвят (червен или черен). Има следните свойства:
 - Всеки връх е или червен, или черен
 - Коренът винаги е черен
 - Листата винаги са черни
 - Ако даден връх е червен то двата му наследника са черни
 - Всеки червен възел има черен родител
 - Всички пътища от произволен връх до произволно листо от поддървото съдържат еднакъв брой черни върхове

Тема 6

- Алгоритъм за търсене – Процес на преглеждане на съвкупност от данни (която може да бъде подредена или не) с цел откриване на конкретен елемент. Търсенето се дели на вътрешно (когато всички данни са разположени в оперативната памет) и на външно (когато данните или част от тях се намират на външен носител). Максималната сложност на алгоритмите за търсене е $O(n)$ т.е. той е линеен.
- Търсене в не подреден масив – Нарича се последователно (линейно), защото нямаме предварителна информация къде може да е елементът. Единственият начин е да проверяваме елементите един по един от началото към края. Има два метода като всеки един от тях е с линейна сложност:
 - Класическо последователно търсене – с помощта на цикъл обхождаме масива и на всяка стъпка извършваме две проверки – дали сме стигнали края на масива и дали текущият елемент е равен на търсения.
 - Търсене с бариера – Изкуствено добавяме търсения елемент в края на масива, за да сведем проверката до една – текущия елемент равен ли е на последния (търсения). Когато цикълът спре и индекса е по малък от този на който сме записали търсената стойност сме намерили реален елемент, ако е равен на последния индекс – такъв елемент не съществува.
- Търсене в подреден масив – Процес на откриване на елемент в структура от данни, където елементите са предварително подредени по големина, което позволява оптимизиране на процеса чрез прекъсване на търсенето, когато стане ясно, че елемента не може да съществува в останалата част. Има три метода:
 - Линейно търсене в подредена таблица – Започваме да проверяваме елементите един по един от началото, като на всяка стъпка има една от три възможности: текущият елемент е равен на търсения и търсенето приключва, текущият елемент е по голям от търсения следователно търсенето приключва с резултат неуспех или текущият елемент е по малък от търсения и продължаваме с процеса. Този метод подобрява бързината при неуспешно търсене.
 - Последователно търсене с преподреждане – Той променя масива докато търси. Целта му е най-често търсените елементи да се преместят в началото на масива, за да ги намираме по бързо (самоорганизиращ се списък)
 - Двоично търсене – Работи на принципа на последователното разполовяване на интервала на търсене, докато елемента не бъде намерен или интервала не стане празен. Алгоритъмът работи само ако масива е сортиран в противен случай е неприложим. Най-напред намираме средната на текущия интервал и сравняваме средния елемент с търсената от нас стойност. В първия случай го намираме, във втория търсения от нас елемент е по-малък (т.е. трябва да е в лявата половина) или в третия случай е по-голям и трябва да е в дясната половина. Повтаряме стъпката с разделянето на интервала и проверката докато не намерим търсената стойност или докато началото на масива не стане по голям от края.
- Хеш – Стойност с фиксирана дължина, получена чрез хеш функция върху данни с произволна дължина, като еднакви входни данни винаги дават еднакъв хеш резултат. Той не е за съхранение на информация, а за нейното бързо разпознаване или

адресиране. Лесно сверим е, защото обикновено се ползват за търсене на дубликати между обектите. Също така е и компактен, когато е малък по размер може да се мести с минимален разход на време и не заема много място.

- Хеширане – Процесът на прилагане на хеш функция върху даден обект (данни), при който той се преобразува в хеш стойност с фиксирана или ограничена дължина, с цел бързо сравнение, адресиране или разпознаване на обектите.
- Хешираща функция – Функция, която приема като аргумент обект и връща неговия хеш. Логиката е различна в зависимост от обекта, който хешираме
- Хеш таблица – Структура от данни за съхранение на елементи, при която стойността на елемент се идентифицира чрез ключ, а позицията му в таблицата се определя чрез хеш-функция, която преобразува ключа (стойността) в индекс.
- Колизия – Ситуация при хеширане, при която два или повече различни ключа се преобразуват от хеш-функция в една и съща позиция на хеш таблицата, което налага използването на допълнителен механизъм за съхранение или разрешаване на конфликта.
- Затворено хеширане – Метод за справяне с колизии, при който всеки елемент се съхранява директно в самата хеш таблица, без използване на външни структури. При възникване на колизия алгоритъмът търси друго свободно място. Прилагат се следните подходи:
 - Линейно пробване – Ако изчисленият индекс е зает, алгоритъмът започва да проверява последователно следващите позиции в таблицата една по една, като търсенето продължава до намирането на първото свободно място където новият елемент се записва
 - Двойно хеширане – Техника при която в случай на колизия се използва втора напълно различна хеш функци, като резултатът от нея определя стъпката с която да се прескочи броя позиции като разстояние, за да се потърси следващото свободно място
 - Рехеширане – Процес на увеличаване на капацитета на таблицата, когато тя се запълни прекалено много. Създава се нова по-голяма таблица и всички елементи се преизчисляват и преместват в нея.
 - Разширено хеширане – Динамичен метод, при който хеш-таблицата не е с фиксиран размер, а може да расте автоматично при нужда
- Отворено хеширане – Метод за справяне с колизии, при който елементите не са ограничени само до клетките на основната таблица, а се съхраняват във външни структури от данни. Всеки индекс в таблицата служи като начална точка, която води към колекция от всички елементи, хеширани до този конкретен адрес.
 - Допълнителна памет за колизия – Таблицата се разделя на две зони: основна за първоначалните записи и специална резервна зона при препълване. Когато възникне колизия в основната част елемента се пренасочва и записва в резервната
 - Списък на препълванията – Метод, при който всеки индекс на таблицата е прикачен свързан списък. Когато възникне колизия, новият елемент просто се добавя като пореден възел в края на списъка съответстващ на този индекс.

Тема 7

- Сортиране – Процес, при който последователност от n на брой елемента се подрежда в строго определен ред. То цели да улесни бъдещи операции (лесно и бързо търсене на конкретни елементи). Едно сортиране наричаме устойчиво когато еднаквите елементи не променят взаимното си разположение след подреждането. Според това къде се намират данните по време на процеса, сортирането се дели на външно и вътрешно:
 - Външно – Данните се намират на външен носител и достъпът до елементите не е пряк
 - Вътрешно – Данните се намират в оперативната памет и имаме пряк достъп до всеки елемент. Само по себе си вътрешното сортиране се дели на частни алгоритми (сортиране на елементи със стойности от определен тип и в определен диапазон) и общи / универсални алгоритми (приложими за елементи от произволен тип, стига да могат да се сравняват)
- Цифрова сортировка – Частен алгоритъм за сортиране на цели числа в зададен диапазон. Осъществява се чрез ключ (само стойност) или по ключи номер (стойност и уникален идентификатор):
 - По ключ – Този метод обикновено се използва за едномерен масив от цели числа, които искаме да подредим по големина. Първо създаваме помощен масив, чийто размер отговаря на най-голямата стойност в оригиналния масив + 1, защото в помощния масив индексите играят ролята на самите числа. В началото този помощен масив е запълнен само с нули. След което обхождаме основния масив и взимаме стойността на текущото число и увеличаваме стойността на съответстващия му индекс в помощния масив с 1 (индексите от помощния масив служат като брояч колко пъти ще срещнем определено число). Накрая обхождаме помощния масив и ако стойността на определен индекс е различна от 0 трябва да запишем номера на индекса толкова пъти колкото е числото в индекса.
 - По ключ и номер – Създаваме помощен масив като клетките му играят ролята на указатели към списъци (стекове или опашки). Обхождаме основните данни, като всеки запис е съставен от ключ и номер. Взимаме номера и го добавяме в списъка, който съответства на ключа от записа в помощния масив. Накрая обхождаме помощния масив и за всеки индекс, който има списък с елементи към него го извеждаме и записваме в крайния масив.
- Лексикографски ред – Правилото, според което символите се подреждат по азбучен ред и думите се сравняват отляво надясно, символ по символ. Лексикографската сортировка е алгоритмичната реализация на това теоритично правилно
- Лексикографско сортиране – Метод за подреждане на думи (символни низове) по азбучен ред. Две думи се сравняват най-напред по първия символ и ако той е еднакъв по втория, ако и той е еднакъв по третия и така докато не се открие различен. Думата, чийто символ на тази позиция е по-малък според азбуката стои по-напред в реда. Тя бива два вида:
 - На думи с фиксирана дължина – Алгоритъм за сортиране на думи, при който всички думи имат еднакъв брой символи (дължина) и думите се състоят от

символи от азбука с x на брой символи, които могат да се съдържат в нея.

Сортирането се прави на няколко стъпки, като на всяка думите се подреждат само по един символ и резултатът от предишните стъпки се запазва. Алгоритъмът извършва толкова обхождания, колкото е дължината на думите като се започва от най-десния символ и се върви наляво, за да може ако два или повече елемента имат еднакъв символ техният относителен ред да се запази. При всяко обхождане се създават x на брой опашки (по една за всеки символ от азбуката) и думите се разпределят в опашките според символа в текущата позиция, след което се събират обратно в реда на опашките

- На думи с променлива дължина – Алгоритъм за сортиране на думи с различна дължина. Тук възниква проблемът, че не всички думи имат символ на дадена позиция. Затова съществуват два варианта:
 - Допълване с празни символи – Така всички думи се изравняват по дължина спрямо най-дългата дума като се допълват с празен символ в дясно по-късите думи и се прилага алгоритъма за фиксирана дължина
 - Сортиране по дължина – При този вариант думите първо се сортират по дължина в намаляващ ред, след което се извършва лексикографското обхождане от дясно наляво и при всяко обхождане участват думите, които имат символ на тази позиция. При първото обхождане участват само най-дългите думи, при следващото още думи като по-късите думи влизат в сортирането когато имат символ за сравнение

- Видове сортиране – Сортирането може да се раздели на няколко групи от методи чрез:
 - Вмъкване – Взима се пореден елемент и се поставя на точното му място в подредбата
 - Размяна – Сравняване на двойки елементи като местата им се разменят, ако не са в правилния ред
 - Избор – Всяка стъпка преглежда всички останали числа и избираме най-малкото или най-голямото, слагайки го на съответната му позиция
 - Сливане – Метода разделя данните на по-малки сортирани групи, които накрая се обединяват в една обща подредена последователност
 - Разпределение – Елементите не се сравняват един с друг а се разпределят в различни групи според стойността им или цифрите им. Накрая се събират обратно подредени
- Оценка на ефективността (анализ на сложността на метода за сортиране) – Ефективността на алгоритмите за сортиране се определя чрез броя сравнения (C) и броя премествания (M) на елементите
- Сортиране чрез просто вмъкване – Алгоритъм, при който масивът се разглежда като разделен на две части: лява (вече сортиран) и дясна (несортиран). Процесът се състои в последователно извличане на първия елемент от несортираната част и вмъкването му на съответната позиция в сортираната част, като всички по-големи елементи се изместват с една позиция надясно, за да освободят място. Методи отнасящи се към тази група сортиране са:

- Двоично вмъкване – Оптимизация на простото вмъкване с разликата, че правилното място се намира чрез двоично търсене, а не с последователно обхождане
- Сортиране на Шел – Вместо да сравнява съседни елементи сравнява такива които са на определено разстояние един от друг, наречено стъпка. С всяко следващото преминаване през масива стъпката се намалява докато не стане 1. Най-напред се избира начална стъпка, която е по-малка от броя на елементите. Елементите се групират мислено в две гру. В една попадат елементи които са на съответното определено разстояние един от друг и всяка една от тези групи се сортира поотделно чрез метода на прякото вмъкване. След това се избира ново по-малко разстояние и отново групираме и сортираме по нея. Процесът се повтаря докато разстоянието не стане равно на 1
- Сортиране чрез размяна – Основава се на принципа на сравняване на двойки елементи и тяхната физическа размяна. Пример за такъв метод са метода на мехурчето, сортиране чрез клатене и метода на Хоар (бързо сортиране):
 - Метод на мехурчето – Сортиране чрез пряка размяна на два съседни елемента
 - Сортиране чрез клатене – Докато методът на мехурчето върви само в една посока клатенето сменя посоката си при всяка стъпка. Избутва най-големия елемент до края и веднага след това тръгва наобратно и избутва най-малкия елемент в началото.
 - Бързо сортиране – Алгоритъм от типа разделяй и владей, който работи чрез избор на един елемент, наречен осов и разделяне на масива на две части спрямо него. В лявата част стоят всички елементи по-малки от осевия, в дясната по-големите от него. След разделянето метода се прилага рекурсивно.
 - Алгоритъм на Хоар – избира се осов елемент и се стартират два показалеца (индекси). Единият тръгва от ляво надясно търсейки елемент по голя или равен на осевия а, другия от дясно наляво в търсене на по-малък или равен и когато открият си разменят местата. Процеса се повтаря докато индексите се разминат.
- Библиотечната функция qsort –Името ѝ идва от Quick Sort и се използва за сортиране на масиви от произволен тип данни. В повечето случаи тя реализира алгоритъма за бързо сортиране или оптимизирания му вариант. Това е универсална функция и работи с байтове като изисква 4 конкретни параметъра:
 - Указател към началото на масива
 - Броя елементи в него
 - Размера на един елемент
 - compare (Функция за сравнение)

Тема 8

- Сортиране чрез пряка селекция – Това е алгоритъм, който работи „на място“ (в рамките на един масив), като разделя масива на сортирана и несортирана част. При всяка стъпка той обхожда напълно несортираната част, намира елемента с най-малка стойност и го разменя физически с първия елемент от тази несортирана зона, като по този начин разширява подредената част.
- Сортиране чрез извличане – Това е метод, при който се използва допълнителна памет (втори масив или списък). Алгоритъмът намира най-малкия елемент в изходното множество, премахва го („извлича“ го) оттам и го записва последователно в новата структура, като процесът се повтаря до изчерпване на изходните данни.
- Сортиране чрез избор от дърво - Това е метод, който организира елементите като листа на двоично дърво и ги сравнения по двойки от долу нагоре. Победителят (по-малкият елемент) се изкачва към корена, а след като бъде записан в резултата, на неговото първоначално място се поставя специална стойност („безкрайност“), което позволява следващият най-малък елемент да бъде открит бързо.
- Пирамидално сортиране - Това е високоефективен алгоритъм, който организира масива в логическа структура, наречена „Пирамида“ (Heap) – двоично дърво, в което всеки родител е по-голям от своите наследници. Тази структура гарантира, че най-големият елемент винаги се намира на върха (позиция 0), което позволява лесното му вземане и преместване в края на масива, без нужда от допълнителна памет.
 - Най-напред се построява пирамидата – пренареждаме елементите в масива така, че да изпълняват условието за пирамида (всеки родител да е по-голям от децата си)
 - Размяна – вземаме елемента от върха и го разменяме с последния елемент на несортираната част на масива.
 - Възстановяване – тъй като на върха е отишъл по-малък елемент от дъното, структурата е нарушена и трябва да се възстанови като новопостъпилия елемент намери своето място в пирамидата. Тези стъпки се повтарят като всеки път се намалява размера на пирамидата с 1
- Сортиране чрез сливане (Merge sort) – Алгоритъм от типа "Разделяй и владей" (Divide and Conquer), който се основава на обединяването на две или повече предварително сортирани последователности (подмасиви) в една обща подредена последователност. Най-напред се извършва разделянето като изходният масив се разделя на две равни половини. Този процес се прилага рекурсивно, докато не се получат подмасиви, състоящи се от само един елемент. След това започва процес на връщане от дъното нагоре. Единичните елементи се комбинират. Накрая идва ред на сливането като двете сортирани половини се обхождат едновременно. На всяка стъпка се сравняват текущите елементи от двете половини, по-малкият се записва в новия общ масив, а указателят му се премества напред. Процесът продължава, докато всички елементи се обединят в една напълно сортирана структура.
- Четно-нечетно сортиране – Паралелен алгоритъм, представляващ модификация на метода на мехурчето. При него сравненията и размените не се извършват последователно, а се разделят на две независими фази (четна и нечетна), които могат да се изпълняват едновременно за всички двойки. В нечетната фаза се сравняват и при нужда се разменят всички двойки елементи с индекси (нечетен, четен), а в четната фаза се

прави обратното – разменят се всички двойки, които са с индекси (четен, нечетен). Двете фази се редуват, докато масивът бъде напълно сортиран.

- Алгоритъм на Бетчер – Метод за изграждане на сортиращи мрежи (Sorting Networks) за паралелна обработка. Той представлява високоефективна схема за паралелно сливане на два сортирани масива, която не зависи от стойностите на данните, а само от техните позиции. Входните сортирани последователности се разделят на две подгрупи: едната съдържа елементите на четни позиции, а другата – на нечетни. След това се прилага рекурсивно сливане, където двете групи се сливат поотделно чрез рекурсивно извикване на алгоритъма. След сливането на подгрупите се извършва финална стъпка, при която съседните елементи се сравняват и разменят, за да се гарантира пълната подредба на обединената редица.
- Сортиране чрез разпределение – Общо наименование за група алгоритми, които не използват директно сравнение между двойки ключове. Вместо това, те използват стойността на самия ключ, за да определят директно мястото на елемента в паметта или да го разпределят в съответна категория. Процесът включва разпределяне на елементите в множество помощни класове (кутии/buckets) и последващото им събиране, което автоматично води до подредба.
- Паралелно сортиране – Процес на подреждане на данни, при който операциите по сравнение и размяна се извършват едновременно (конкурентно) от множество процесорни елементи, с цел намаляване на общото време за изпълнение спрямо последователните алгоритми. Ефективността на паралелното сортиране зависи от начина, по който процесорите са свързани помежду си:
 - Линеен масив – Процесорите са подредени в редица.
 - Двумерен масив – Процесорите са разположени в решетка (матрица).
 - Хиперкуб – Многомерна свързаност, осигуряваща бърз обмен на данни между отдалечени процесори.
- Външно сортиране – Съвкупност от методи за обработка на файлове с голям обем, които надвишават капацитета на оперативната памет (RAM). Данните се съхраняват на външни носители, а достъпът до тях е последователен и значително по-бавен от този при вътрешното сортиране. Всички методи за външно сортиране се базират на предварително сортирани части:
 - Пряко сливане – Файлът се разделя на две половини и се извършва сливане на единични елементи, след това на двойки, четворки и т.н., докато се получи подреден файл.
 - Естествено сливане - Методът разпознава и използва съществуващи предварително подредени последователности в изходните данни. Сливат се цели серии, вместо фиксирани блокове, което прави процеса по-ефективен при частично подредени файлове.
 - Балансирано многоходово сливане - При този метод се използват равен брой входни и изходни помощни файлове. Ако имаме $2k$ устройства, k се използват за четене (вход), а другите k – за запис (изход). След всяко преминаване ролите на входните и изходните устройства се разменят.
 - Многофазно сливане – Оптимизиран вариант на многоходовото сливане, който пести хардуерни ресурси. Вместо $2k$ устройства, използва само $k+1$ устройства. Разпределението на сериите по устройствата не е равномерно.

Тема 9

- Точен алгоритъм – Алгоритъм гарантиращ, намирането на най-оптималното (най-доброто) решение, като изследва всички възможни варианти.
Недостатъците му, са че може да е толкова сложен, че да е невъзможно да се състави или ако бъде съставен то времето за изпълнението му нараства с нараства с размера на задачата, правейки го неизползваем в практиката. Затова се използват евристични (приблизителни алгоритми)
- Евристичен алгоритъм – Подход, който не гарантира най-точното решение, но са много прости и лесно реализуеми. Те дават добри, но не винаги оптимални решения. Вместо да гледа цялата картина, алгоритъмът избира решение на базата на локален критерий (какво изглежда най-добре в момента), надявайки се то да доведе до добър край.
 - Задача за търговски пътник – Целта на задачата е търговски пътник трябва да обходи N на брой градове и да се върне в началото, като измине най-краткия път. Започваме от начален град и виждаме кои са съседните градове, които още не сме посетили. Избираме този който е най-близо и повтаряме докато обиколим всички градове. Този алгоритъм не гарантира най-краткия възможен път в света, но дава приемлив маршрут много бързо.
- Алчни алгоритми – Те са подклас на евристичните алгоритми и се насочват към най-добрия за момента избор. На всяка стъпка алгоритъма прави избор, който е най-добър (оптимален) за момента, без да се интересува как ще се отрази на бъдещето. Недостатъкът им е, че не винаги гарантират правилното решение на задачата, макар че намереното винаги е близко до оптималното.
 - Задача да египетските дробни – Древните египтяни са представляли всяка дроб като сума от дробни с числител 1 (напр. $\frac{1}{2}, \frac{1}{3}, \frac{1}{4} \dots$). За да представим дробта P/Q първо намираме най-голямата възможна дроб от вида $1/A$, която е по-малка или равна на P/Q . След това изваждаме от P/Q . И повтаряме процеса за остатъка, докато не стане 0.
 - Задача за съчетаване на дейности – Имаме една зала и списък с лекции, всяка с начален и краен час. Искане да проведем максимален брой лекции, без да се застъпват. Първо сортираме всички лекции по техния краен час (коя свършва най-рано). Избираме първата (тази, която освобождава залата най-скоро). Пропускаме всички следващи, които започват, преди текущата да е свършила (застъпващите се). Избираме следващата възможна и повтаряме.
- Вероятностен алгоритъм – За разлика от детерминирани (точните) алгоритми, ако пуснем вероятностен два пъти с едни и същи входни данни, той може да даде различен резултат или да стигне до него по различен път. тези алгоритми използват случаен избор (генератор на случайни числа) в хода на работата си. Те биват няколко вида:
 - Числен – Използва се за приближено пресмятане.

- Монте Карло – Винаги дава някакъв отговор, но има малка вероятност да е грешен
- Лас Вегас – Никога не дават грешен отговор, той винаги е 100% верен, но няма гаранция кога ще бъде намерен. Може да отнеме много време, да стане веднага или изобщо да успее да намери решение.
- Генетичен алгоритъм – Метод за търсене на решения, който имитира биологичната еволюция и естествения подбор, оцеляват най-добрите. Създава се множество от случайни решения (начална популация) и се оценява кои индивиди са най-добри (пригодност). Най-добрите стават родители и се комбинират, за да създадат нови решения (насленици), а лошите решения отмират и така процеса се повтаря
 - Задача за търговски пътник – вместо да търсим път с математика, създаваме много случайни маршрути (популация) и взимаме два кратки маршрута (родители), които комбинираме като вземаме част от маршрута на единия и го слепваме с част от този на другия родител. За да не зациклим понякога случайно разменяме два града в маршрута и след много поколения (итерации) се получава маршрут, който е много близък до най-краткия възможен.

Тема 10

- Методи за съставяне на алгоритми – Това са системни подходи и стратегии за проектиране на компютърни програми, които помагат сложните задачи да бъдат разбити на управляеми части. Има две основни групи за формално съставяне на алгоритми:
 - Поредово програмиране – Използва се при по-прости задачи, където инструкциите се пишат последователно.
 - Специални методи и подходи – Целта е повишаване на ефективността при проектиране на реализация на сложни проекти.
- Видове методи за съставяне на алгоритми:
 - Структурно програмиране – Разделяне на програмата на логически блокове.
 - Обектно-ориентирано програмиране (ООП) – Моделиране на проблемите чрез обекти, съдържащи данни и методи.
 - Метод на частните цели – Стратегия за постигане на междинни резултати
 - Метод „Разделяй и владей“ – Разбиване на сложна задача на по-малки подзадачи, които се решават поотделно.
 - Метод с връщане назад (Backtracking) – Метод за намиране на решения чрез систематично търсене и връщане назад при достигане на „задънена улица“.
 - Евристични алгоритми – Алгоритми, даващи приблизително, но бързо решение
 - Рекурсия – Програмен метод, при който една функция извиква сама себе си за решаване на по-малка инстанция на същия проблем.
 - Динамично програмиране – Метод със запомняне на междинни резултати за избягване на повторни сметки
- Структурно програмиране – Метод за съставяне на алгоритми и програми, който има за цел да улесни процесите на кодиране, тестване, отстраняване на грешки и модифициране на програмите. Неговите цели са:
 - Да се осигури дисциплина в програмирането.
 - Да се подобри, четимостта на програмите.
 - Да се повиши ефективността на проверката на програмите.
 - Да се повиши надеждността и улесни поддръжката.
 - Да се намалят времето и стойността на разработката.
- Принципи на структурното програмиране:
 - Принцип на абстракциите – Фокусиране върху съществените характеристики на задачата, като се игнорират сложните детайли на по-ниско ниво.
 - Принцип на формалността – Използване на строги правила и логически подход за доказване на правилността на алгоритъма, вместо разчитане на интуиция.
 - Принцип „Разделяй и владей“ – Разбиване на големия проблем на по-малки и независими подпроблеми, които са по-лесни за управление.

- Принцип на йерархичното подреждане – Подреждане на елементите на системата в нива (слоеве), където всяко ниво стъпва върху по-долното.
- Метод с връщане назад (Backtracking) - Това е метод за намиране на решение чрез принципа „проба-грешка“. Той проверява систематично възможните варианти един по един. Използва се, когато търсим отговор на въпроси като: „Има ли начин да се направи това?“ или „Намерете всички възможни начини“. Процесът се гради стъпка по стъпка, като най напред избираме една възможност (вариант) за продължение и я добавяме към текущото ни частично решение. След това проверяваме дали този избор е валиден (дали не нарушава правилата). Ако е валиден, продължаваме напред към следващата стъпка (влизаме по навътре), но ако не е валиден (или стигнем до задънена улица) се връщаме назад. Отказваме се от този избор (изтриваме го) и се връщаме с една стъпка назад в предишното състояние и пробваме следващия възможен вариант
- Рекурсия – Рекурсията е метод в математиката и програмирането за дефиниране на функция или езикова конструкция чрез самата себе си.
- Метод за съставяне на алгоритми чрез рекурсия – Това е подход, при който решението на даден проблем зависи от решенията на по-малки инстанции на същия проблем. Програмата (функцията) извършва обръщение към себе си, за да извърши част от работата. Всяка рекурсивна функция задължително се състои от три компонента: действие (тяло на функцията), условие (задължителна проверка, която прекъсва безкрайното самоизвикване) и рекурсивно обръщение (извикване на самата функция). Процесът се разглежда като движение в две посоки рекурсивно спускане (напред) – влизане навътре в нови и нови извиквания и рекурсивно завръщане (назад) – връщане към предходното ниво след достигане на дъното.
- Форми на рекурсивните функции – В зависимост от това къде спрямо рекурсивното изискване се извършва същинското действие. Различаваме три форми:
 - Действие при спускане – Първо действаш, после предаваш нататък. Полезното действие се извършва ПРЕДИ рекурсивното извикване. Функцията първо си свършва работата за текущото ниво, и чак след това извиква себе си за следващото ниво.
 - Действие при връщане - Първо слизаш до дъното, и вършиш работа чак на връщане. Полезното действие се извършва СЛЕД рекурсивното извикване. Функцията веднага извиква себе си (влиза навътре), изчаква да се стигне до дъното и резултатите да започнат да се връщат, и чак тогава (на излизане) извършва своята обработка.
 - Комбинирана – Правиш нещо на влизане, и довършваш нещо друго на излизане. Има действия както ПРЕДИ, така и СЛЕД рекурсивното извикване. Част от работата се върши при "спускането" надолу, следва рекурсивното извикване, а останалата работа се довършва при "връщането" нагоре.

- Ефективност на рекурсията
 - Недостатъци (Защо може да е неефективна):
 - При всяко обръщение се заделя памет в стека за параметрите и локалните променливи.
 - Ако дълбочината на рекурсията е твърде голяма или липсва коректно дъно, системния стек се препълва.
 - Обикновено са по бавния от итеративните (цикличните) решения и по трудно се тестват.
 - Предимства (Защо се използва):
 - Повишава ефективността на програмиста – сложни задачи се решават с малко и ясен код.
 - Програмите са по ясни, кратки и разбираеми.
- Метод Разделяй и владей – Идеята на метода е сложната задача да се раздели на няколко по прости подзадачи, които се атакуват по лесно поотделно и чието решение позволява лесно конструиране на решението на изходната задача. Най-напред се извършва разделянето. Разбива се изходната задача на няколко подзадачи (най-често на две). След това идва владееенето. Всяка от задачите се решава поотделно. И накрая въз основа на решенията на малките подзадачи се сглобява (конструира) решението на голямата задача.
- Динамично програмиране – Метод за съставяне на алогоритми, при който резултатите от вече решените подзадачи се запомнят (съхраняват се в паметта), за да се избегне повторното им пресмятане. Основното му предназначение е решаването на оптимизационни задачи (търсене на най-изгодно, най-евтино, най-кратко решение).
 - Задача за стабилните бракове – Дадени са две групи с равен брой участници, множество А (мъже) и множество В (жени). Всеки участник има личен списък с предпочитания, в който е подредил всички представители на другия пол от най-желан към най-малко желан. Целта е да се образуват n на брой двойки (бракове). Стабилен брак е разпределение на двойките, при което не съществува ситуация, в която един мъж е женен за определена жена, но харесва друга повече от нея, както и обратното. Ако такава ситуация съществува, бракът е нестабилен, защото те биха напуснали партньорите си, за да бъдат заедно. За решаването на задачата най напред използваме алгоритъм, който търси партньорка за поредния мъж. Мъжът прави предложение на жената според своя списък с предпочитания. След това се проверява дали изборът е приемлив, тоест дали създава стабилност спрямо текущото състояние. Ако е приемлив фиксираме брака и минаваме към следващия мъж, но ако стигнем до задънена улица, тоест не можем да намерим стабилен вариант за някого отменяме последния брак, връщаме се назад и пробваме следващата жена в списъка. Този процес гарантира намирането на стабилно множество.

Тема 11

- Основен принцип на програмното управление – Работата на компютъра се управлява от програма.
- Програма – Последователност от действия (инструкции), които се извършват от компютърната система при обработка на входни данни с цел получаване на резултати (изходни данни). Програмите биват:
 - Системни – Служат за управление на ресурсите на компютъра (операционна система или драйвери).
 - Приложни – Създават се за обслужване на конкретни нужди на потребителя.
- Бройна система – Начинът, по който записваме и четем числата. Тя определя какви символи (цифри) използваме и каква стойност има всяка позиция в едно число. Биват два вида:
 - Непозиционни – При тях всяка цифра има една и съща стойност независимо къде се намира в числото (римски числа – X винаги е 10). Проблемът, е че за големи числа записът става много дълъг или сложен и дтрудно се смята
 - Позиционни – Стойността на цифрата зависи от мястото ѝ в числото. Този тип бройна система прави записа и изчисленията лесни и удобни
- Основни позиционни системи:
 - Десетична – Основата е 10 цифри (0 – 9).
 - Двоична – Основата е 2 цифри (0 и 1).
 - Осмична - Основата е 8 цифри (0 – 7).
 - Шестнадесетична - Основата е 16 цифри (0 - 9 и букви A-F, където A=10, B=11..F=15.
- Преобразуване на числата
 - От произволна бройна система (с основа S) към Десетична – Всяко число в позиционна система може да се представи като полином (сума): $A = \sum a_i \cdot S^i$
 - От Десетична към произволна бройна система (Цели числа) – Първо разделяме числото A на основата S, полученият остатък от делението е стойността на текущия най-младши разряд. Частното става новото число A. Повтаряме процеса, докато частното стане 0. Получените остатъци записани в обратен ред дават числото.
 - Преобразуване на правилни дробни - Умножаваме дробната част A с основата S. Цялата част на полученото произведение е стойността на текущия разряд. За ново A взимаме само дробната част на произведението. Повтаряме, докато дробната част стане 0 или достигнем желаната точност
- Аритметични действия в бройни системи - Аритметичните действия в двоичната система се правят по същите правила, както в десетичната, но с две цифри – 0 и 1. Разликата е само в това кога се получава пренос или заемане.
 - Двоично събиране:
 - $0 + 0 = 0$

- $1 + 0 = 1$
- $0 + 1 = 1$
- $1 + 1 = 0$ (и пренос 1 към старшия разряд)19.

○ Двоично изваждане:

- $0 - 0 = 0$
- $1 - 0 = 1$
- $1 - 1 = 0$
- $0 - 1 = 1$ (със заем от старшия разряд).

○ Осмично събиране - Пренос 1 се получава, когато сумата на цифрите ≥ 8 .

○ Шестнадесетично събиране – Пренос 1 се получава, когато сумата е ≥ 16

○ Умножение – Извършва се чрез последователно умножаване (което при двоични числа е просто преписване на множимото или записване на нули) и изместване (шифтане) наляво или надясно

○ Деление - Прилагат се правилата за деление, както в десетична система, но е по-просто, защото цифрата на частното може да е само 0 или 1 ($111001 : 110 = 1001$ с остатък 11).

- Прав код – Най-естественият начин за записване. При него числото се представя като комбинация от знак и абсолютна стойност (модул). Положителните числа се записват със знак 0, следван от двоичния запис на числото, а отрицателните числа със знак 1, следван от двоичния запис на числото. Недостатъците са, че аритметичните операции като събиране и изваждане със сложни, защото изискват проверка на знаците и извършване на различни действия според тях. Може да има две нули като +0 и -0, което затруднява сравнението.
- Обратен код – Използва се за опростяване на операциите с отрицателни числа. За положителните числа съвпада с правя код (знак 0 плюс числото), но за отрицателните числа се получава чрез инвертиране (обръщане) на всички битове на положителното число (0 става 1, 1 става 0), включително знаковия бит, (който става 1).
- Допълнителен код – Най-използваният код в съвременните компютри за извършване на аритметични операции. Той позволява изваждането да се сведе до събиране. Положителните числа съвпадат с правия и обратния код (знак 0 + числото), а при отрицателните числа първо се намира обратния код на числото и след това към получения резултат се прибавя 1 в най младшия разряд.

Аритметичните действия са:

- Събиране – Числата се събират бит по бит, включително и знаковите битове. Ако при събирането на знаковите битове се получи пренос, той се игнорира.
- Препълване – При събиране на две числа с еднакъв знак. Ако резултатът има противоположен знак е настъпило препълване. Резултатът не се побира в разрядната решетка.

- Двоично-десетични кодове - Това е метод за представяне на десетични числа, при който всяка десетична цифра (0-9) се кодира поотделно с група от 4 двоични бита (тетрада).
- Коректност – Мярка за това дали алгоритъмът изпълнява точно поставената задача за всички допустими входни данни. Различаваме два вида коректност:
 - Частична коректност – Алгоритъмът е частично коректен, ако за всеки вход, за който той приключва работа (спира), резултатът е верен. (Т.е. ако не забие в безкраен цикъл, ще даде верен отговор).
 - Пълна коректност – Алгоритъмът е напълно коректен, ако е частично коректен и се докаже, че той винаги спира (терминира) за краен брой стъпки.
- Методи за проверка на коректността – Съществуват два коренно различни подхода за установяване дали една програма работи правилно (експериментален метод – тестване и аналитичен метод – верификация).
 - Експериментален метод – Процес на изпълнение на програмата с цел откриване на грешки. Избират се конкретни тестови данни за вход и се сравняват получените резултат с очаквания. Целта е да се покаже наличието на грешка. Тестването може да се осъществи чрез метода на черната кутия, Мм.където тестваме само входа и изхода, без да ни интересува как е написан кодът вътре или чрез метода на бялата кутия, където тестващият познава логическата структура на програмата и пише тестове, които да покрият всички възможни пътища на алгоритъма.
- Аналитичен метод – Това е метод за формално математическо доказателство, че програмата съответства на своята спецификация. Тук не пускаме програмата с данни, анализираме кода ѝ като математическа формула.
 - Метод на Флойд – Наричан още метод на индуктивните твърдения, е основният подход за аналитична верификация. Имаме условие, което е вярно преди старта на програмата (входен предикт) и условие, което трябва да е вярно след (нейния край). Трябва да се докаже, че ако входното условие е вярно и инструкциите на алгоритъма се изпълнят, то изходното условие задължително ще бъде вярно. Основна трудност са циклите. Най трудната част при аналитичния метод е доказването на верността на циклите. За тази цел се използва понятието инварианта. Няма формула или правило за автоматично намиране на инварианта. Тя трябва да се налучка или измисли чрез интуицията от човека.
- Инварианта – Условие, което е вярно преди влизане в цикъла, по време на всяко завъртане на цикъла и след излизане от него.